

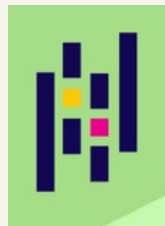


INFORMÁTICA II

Bioingeniería



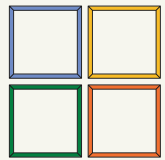
Índice



**Pandas,
Introducción**



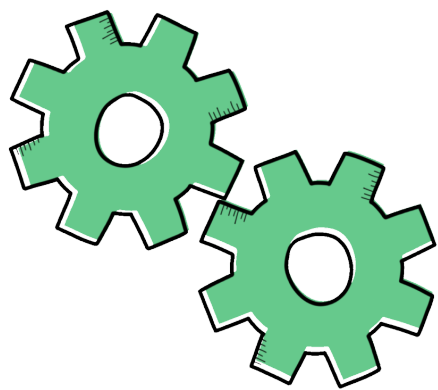
Series

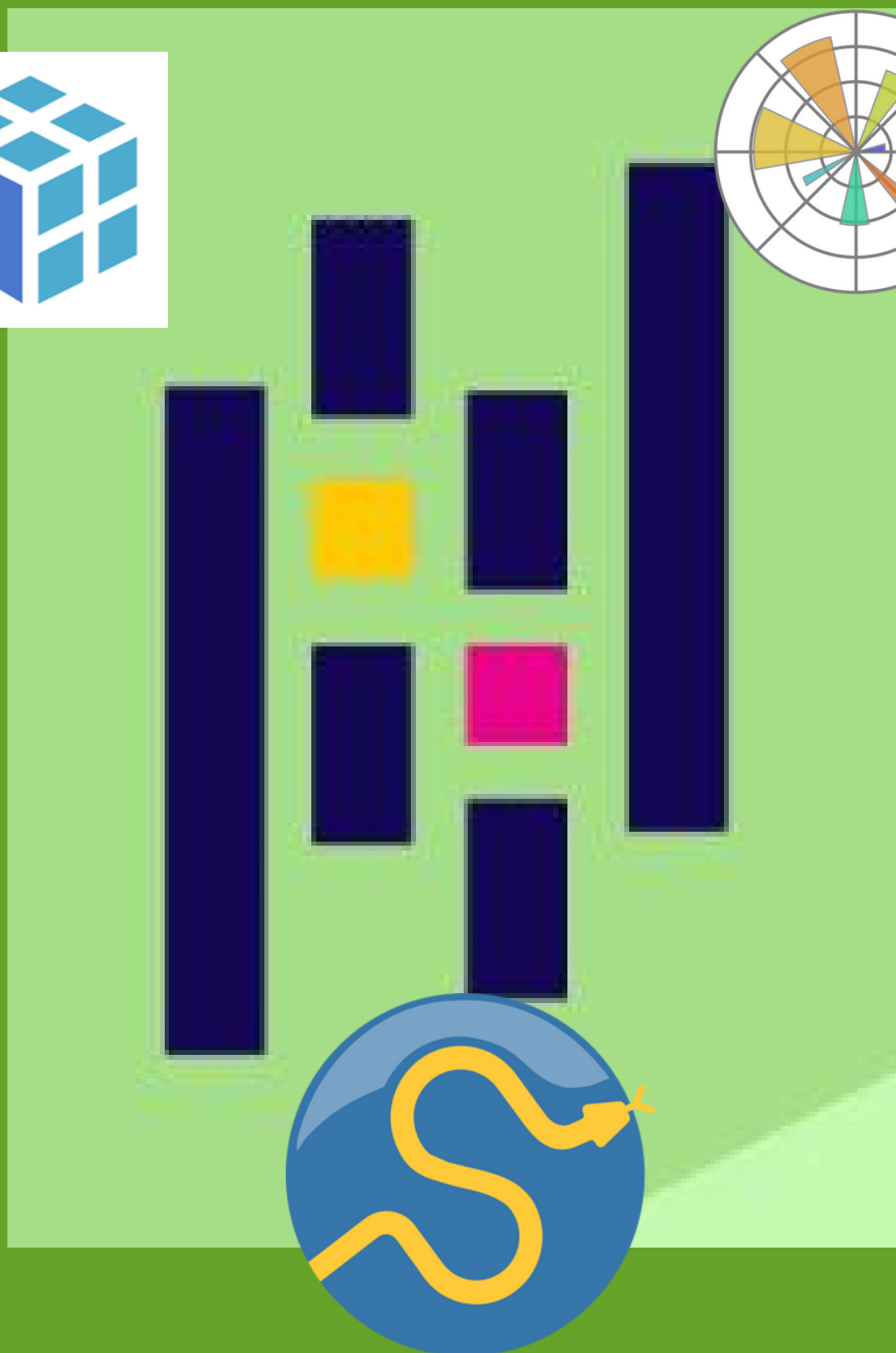


Data Frames



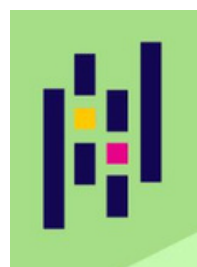
Ejercicios





Unidad 2

Pandas

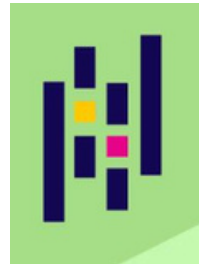


Introducción a Pandas

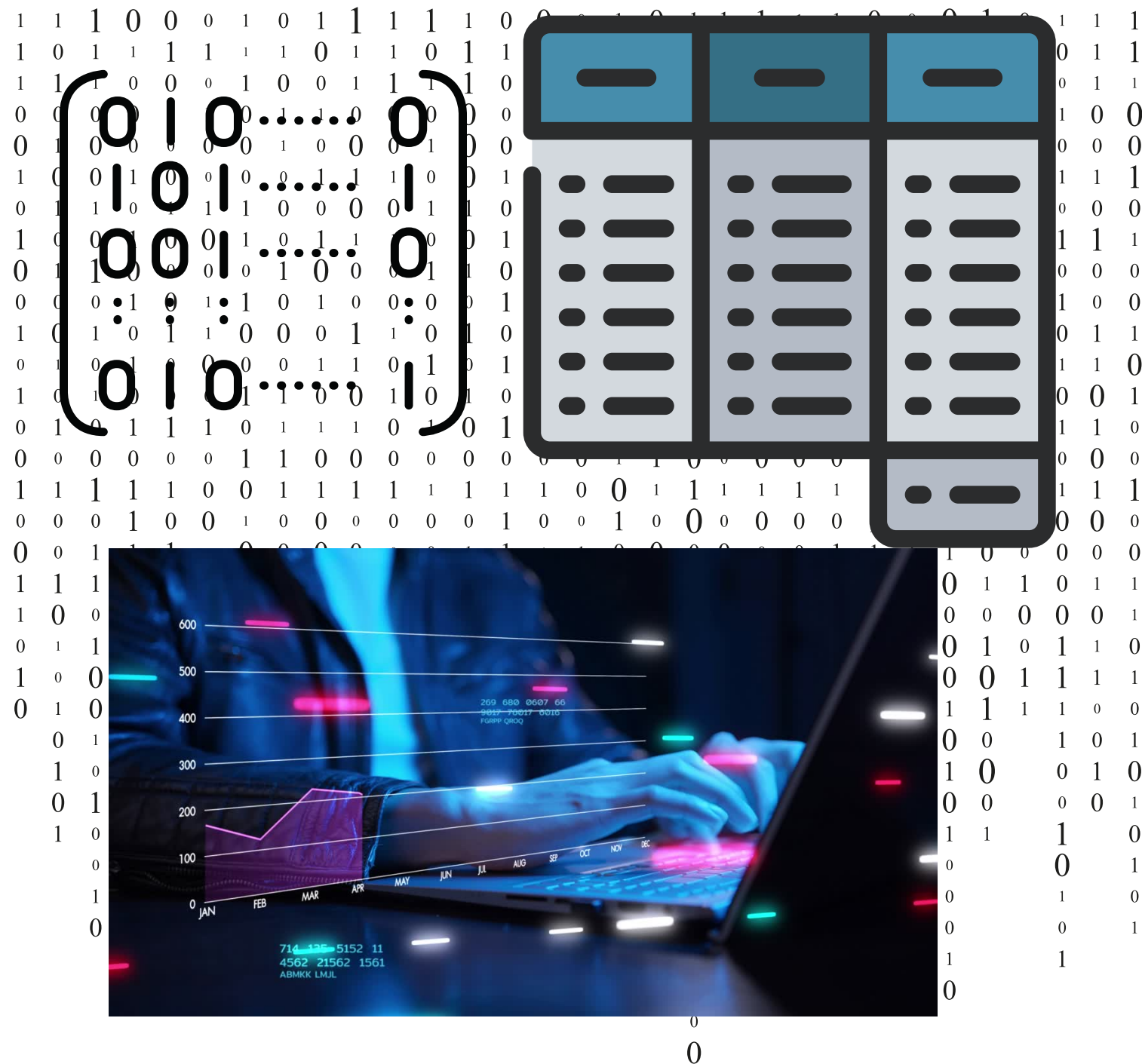


Panel Data

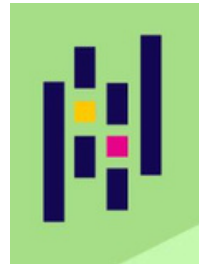
Person ID	Age	Sex	Year	Income
1	27	1	2015	1600
1	28	1	2016	1500
2	42	2	2015	1900
2	43	2	2016	2000
2	44	2	2017	2100
3	34	1	2015	3300



Introducción a Pandas



- Creada en el 2008 por Wes McKinney
- Necesidad que da origen a la iniciativa: manejo de volúmenes de datos en mercados financieros
- Pandas es una librería para manipular y analizar datos de manera eficiente, ágil y fácil.
- Proporciona herramientas poderosas para trabajar con datos estructurados, como tablas o bases de datos. Permite leer, escribir y manipular datos de diferentes formatos, como Json, XML, CSV, Excel, SQL y más.

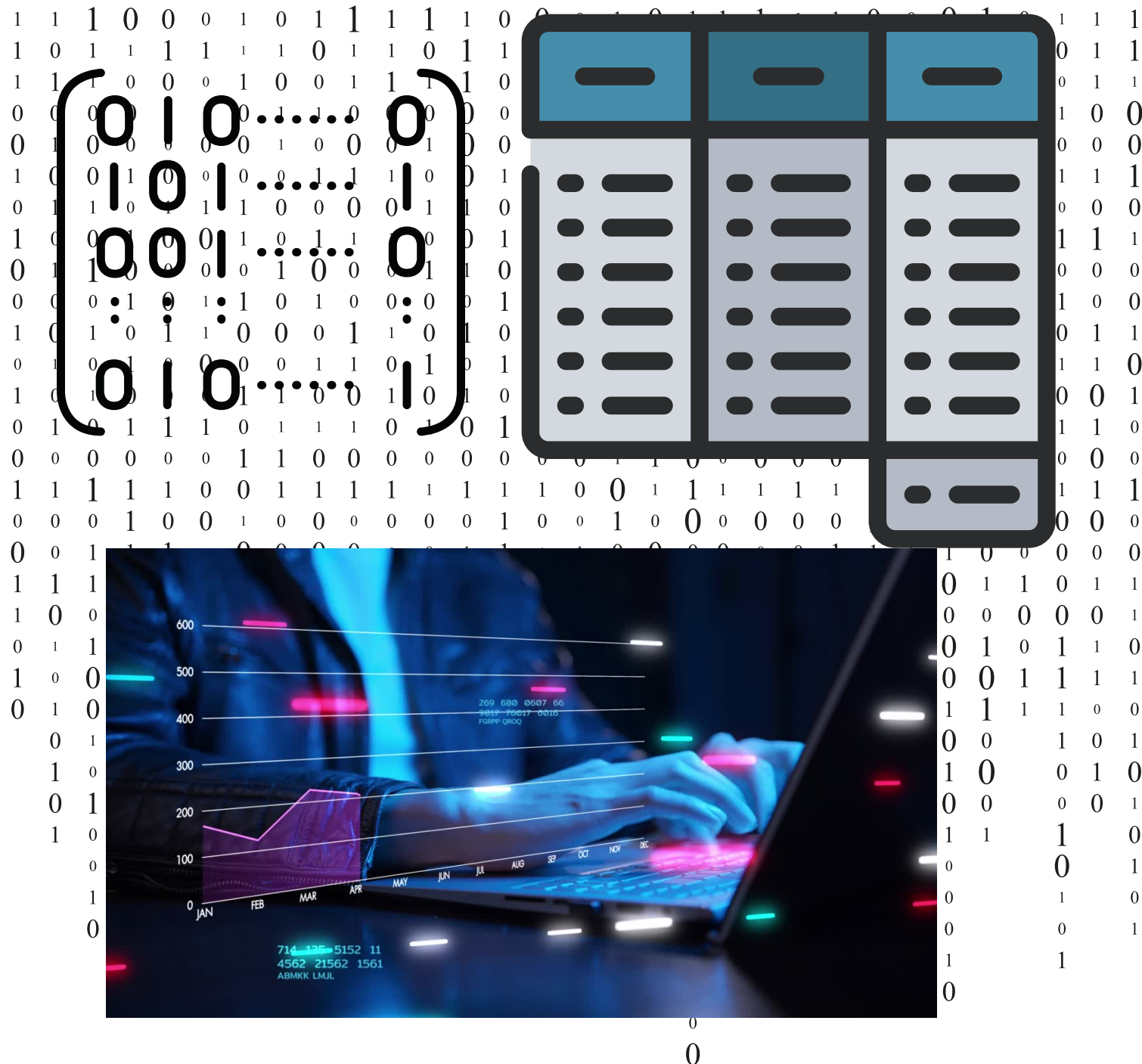


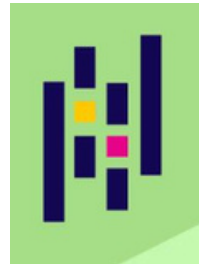
Introducción a Pandas



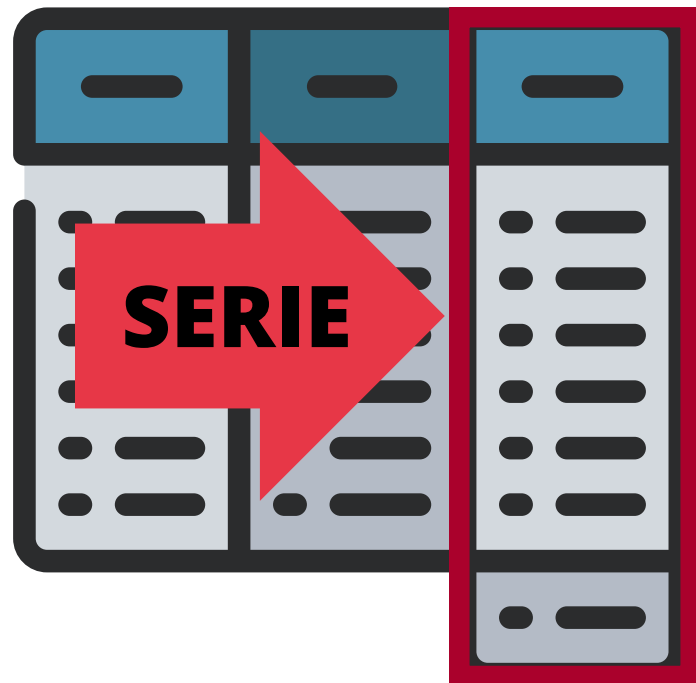
Principales características:

- Estructuras de datos eficientes para el manejo de datos tabulares, como DataFrames y Series
- Funciones para la limpieza, transformación y manipulación de datos
- Herramientas para la unión, fusión y agrupación de datos
- Capacidad para trabajar con datos faltantes o nulos
- La alineación de datos es intrínseca.
- Funciones para la visualización de datos
- El vínculo entre las etiquetas y los datos no se romperá a menos que usted lo haga explícitamente.





Series

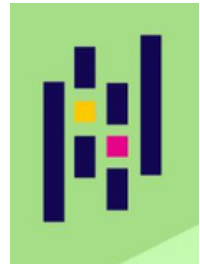


Una serie es una matriz ***etiquetada*** ***unidimensional*** capaz de contener cualquier tipo dato(enteros, cadenas, flotantes, objetos Python, etc.). Pero a diferencia de un vector usual de numpy, este cuenta con una columna o eje de indexación.

Se invoca a través del método ***pd.Series(data, index = índices)*** donde **data** puede ser diferentes tipos de elementos:

- Dicionarios de Python
- Listas de Python
- Un Valor escalar
- Un arreglo de numpy (ndarray)

Y los **índices** valores tipo **str** o **numericos(float, int)** normalmente



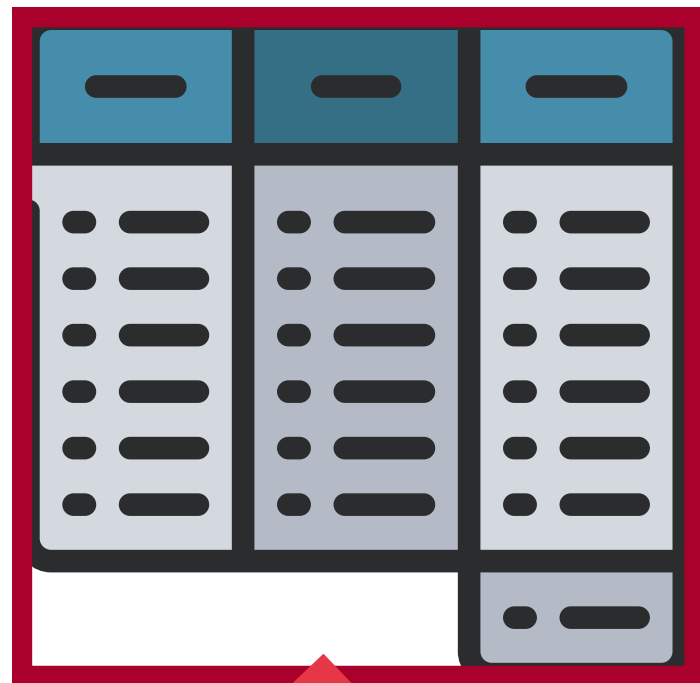
DataFrames



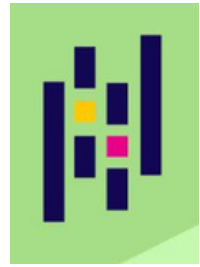
Es una estructura de datos etiquetada, bidimensional, con columnas de tipos potencialmente diferentes. Puedes pensarlo como una hoja de cálculo, o SQL tabla, o un diccionario de **objetos tipo serie**. Generalmente es el objeto utilizado de pandas.

Al igual que Series, DataFrame acepta muchos tipos diferentes de entrada:

- Dicts de ndarrays 1D, listas, dicts o Series
- 2-D numpy.ndarray
- Una Series
- Matriz estructurada (registro txt,cvs, xls, etc..)
- Otro DataFrame



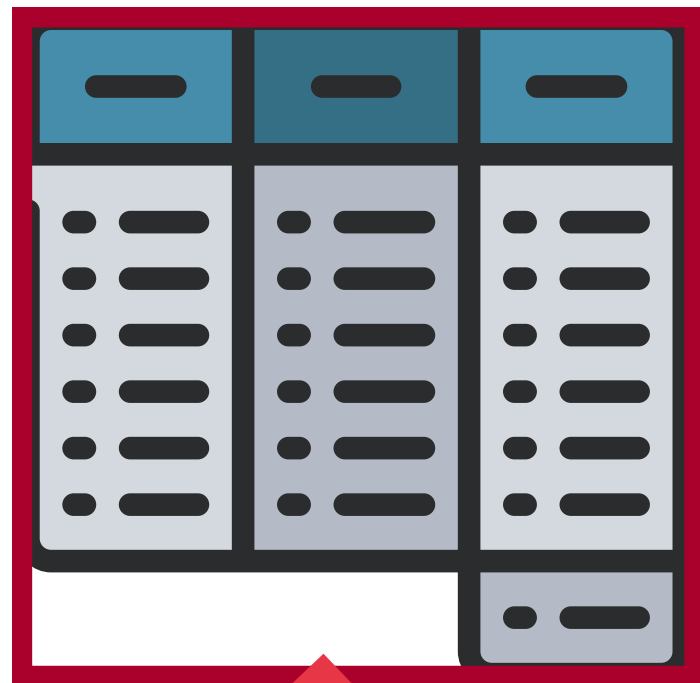
DATAFRAMES



Formatos

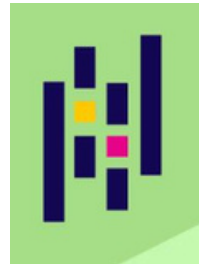


La versatilidad que ofrece Pandas para manipular datos incluye el poder manipular, mas formatos que numpy entre ellos tenemos:



DATAFRAMES

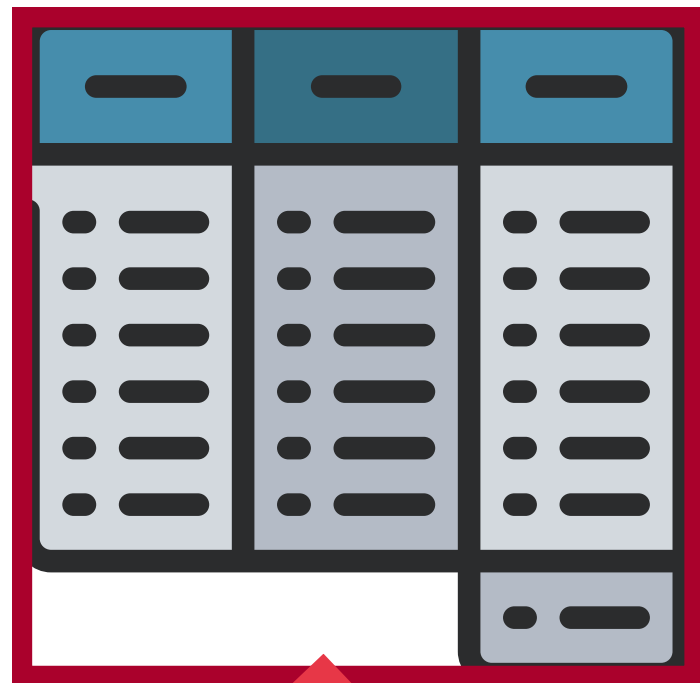
- Excel (**`pd.read_excel("archivo.xlsx ó direccionArchivo")`**)
- CSV (**`pd.read_csv("archivo.csv ó direccionArchivo")`**)
- HDF (**`pd.read_hdf("archivo.hdf ó direccionArchivo")`**)
- Json (**`pd.read_json("archivo.json ó direccionArchivo")`**)
- Parquet (**`pd.read_parquet("archivo.parquet ó direccionArchivo")`**)
- Pickle (**`pd.read_pickle("archivo.pkl ó direccionArchivo")`**)



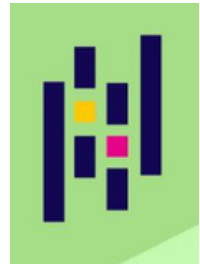
Formatos



Para convertir a dichos formatos



- Excel (**`df.to_excel("archivo.xlsx ó direccionArchivo"), index = False`**)
- CSV (**`df.to_csv("archivo.csv ó direccionArchivo", index = False)`**)
- HDF (**`df.to_hdf("archivo.hdf ó direccionArchivo" key='data', format='table')`**)
- Json (**`df.to_json("archivo.json ó direccionArchivo")`**)
- Parquet (**`df.to_parquet("archivo.parquet ó direccionArchivo")`**)
- Pickle (**`df.to_pickle("archivo.pkl ó direccionArchivo")`**)



Atributos



Al igual que los arreglos de numpy , los Dataframes cuentan con las mismos atributos principales y se llaman igual:

df = pd.read_csv("ruta del archivo")

df.shape

df.size

df.ndim

Adicionalmente cuenta con una serie de metodos que brindan información estadística del DataFrame:

df.index

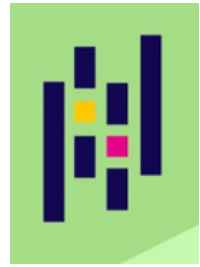
df.columns

df.values

df.describe()

df.info()

df.dtypes



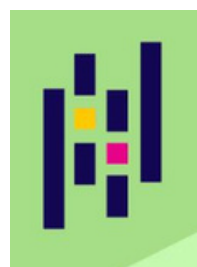
Indexación



En Pandas tambien se puede indexar como las ndarrays de numpy, solo rangos de filas indicando **índice numérico**; [fila1:filan]

Debido a su estructura permite más formas de indexar con la misma sintaxis de un diccionario usando como claves los nombres (labels) de las filas y columnas

Usando métodos como **loc** e **iloc**



Indexación



Usando indexación fila igual a a numpy:

`dataFrame [if:if]`

`dataFrame [ if1:if10:paso]`

`dataFrame [dataFrame[nC] > valor ]`

Usando nombres de columna:

`dataFrame[nC]`

`dataFrame[ [nC1, nC10,..., nCn] ]`

Usando loc y como indexación los nombres de fila y columna:

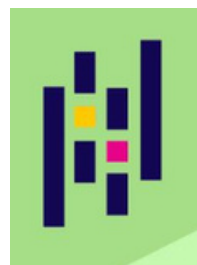
`dataFrame.loc([nf,[nC1, nC10,..., nCn])`

`dataFrame.loc[ [nf1, nf4,...,nfn] , [nC1, nC10,..., nCn] ]`

Usando iloc como indexación los indices:

`dataFrame.iloc[iF1, [iC0, iC11,...iCn]]`

`dataFrame.iloc[[if1, if4,...,ifn], [iC0, iC11,...iCn]]`



Borrado



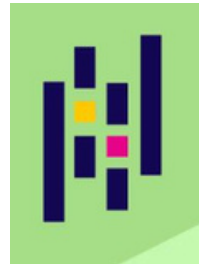
Para borrar elementos en un dataframe , se usa el método **drop** , indicando entre corchetes la columna o columnas que desea borrar, también se debe indicar el axis para el caso de solo borrar una sola columna o fila, adicionalmente cuenta con el argumento **inplace** que si se coloca True ,hará efectivo el cambio en el dataframe.

Tambien es posible pasarle a través de los argumetos columns e index y entre corchetes los elementos a borrar

```
df.drop(['genero'], axis=1, inplace=True)
```

```
df.drop([0,2,4,6]) # Por defecto asume filas
```

```
df.drop(columns= ['id','apellido'],index=[0,2,4,6])
```



Operaciones aritméticas



Las operaciones con Series y DataFrames son iguales que operando np.arrays con la salvedad de que puede implementarse la indexación previamente mostrada, también es posible implementar los metodos propios de numpy para hacer operaciones

`np.sin(df['speed']**2) + 10`
`df['t1']/df['t2']`

```
+    add()
-    sub(), subtract()
*    mul(), multiply()
/    truediv(), div(), divide()
//   floordiv()
%    mod()
**   pow()
```

GRACIAS